ST. ANDREWS INSTITUTE

OF TECHNOLOGY & MANAGEMENT

Gurgaon Delhi (NCR)

Approved by AICTE, Govt. of India, New Delhi Affiliated to Maharshi Dayanand University 'A' Grade State University, accredited by NAAC

Session: 2025 – 2026

Bachelor of Technology

COMPUTER SCIENCE & ENGINEERING

A Practical File

Subject Name – Programming in Java

Subject Code- LC-CSE-327G

Submitted To:
Mr. Parveen Yadav

Submitted by:

NAME:Deepak kumar thakur
SEM:5TH
ROLL NO:233026

St. Andrews Institute of Technology & Management, Gurugram

Department of……

Practical Lab Evaluation Sheet

S.No	Program	Date	CO	Practical Performed (10)	Viva Voce (05)	Attendance (05)	Practical File (05)	Overall (25)	Remarks & Signature
1	Write a program in java to print your details(Name,Rollno,Class, Sem, College Name). And Create theJAR file of this program. with both output i.e .java and .Jar.		CO1						
2	Create a java program to implement stack and queue concepts.		CO1						
3	Write a java package to show dynamic polymorphism and interfaces.		CO2						
4	WAP to implement the concept of String and String Buffer.		CO2						
5	WAP in java to implement the concept of Exception Handling with Five Keywords:- TRY,CATCH,FINALLY,THROW AND THROWS.		CO3						
6	Write a java program to show the concept of multithreading.		CO3						
7	WAP in java to create packages in java.		CO3						

8	WAP in Java to create a Calculator using AWT.		CO4						
9	Create an editor like MS-word using swings.		CO4						
10	Write a java program to show the concept of Collections.		CO4						
11	Write a program on synchronization using mutual exclusive types.		CO3						

Program 1

Write a program in java to print your details (Name, Rollno, Class, See, College Name). And Create the JAR file of this program with both output i.e. java and .Jar

```
public class MyDetails {
   public static void main(String[] args) {

        String name = "Deepak kumar Thakur";
        String rollNo = "233026";
        String clazz = "B.Tech Computer Engineering";
        String sem = "3rd Year";
        String college = "St. Andrews Institute of Technology & Management";

        System.out.println("----- Student Details -----");
        System.out.println("Name      : " + name);
        System.out.println("Roll No.  : " + rollNo);
        System.out.println("Class     : " + clazz);
        System.out.println("Semester  : " + sem);
        System.out.println("College   : " + college);

}

}
```

1) Compile & run on Terminal

Open Terminal and run these commands in the folder containing

MyDetails.java. javac MyDetails.java

This produces
MyDetails.class.
Run the class directly: java
MyDetails

Run the java MyDetails > xf_output.txt command

To print the xf output (execution output)

```
----- Student Details -----
Name       : Deepak kumar Thakur
Roll No.   : 233026
Class      : B.Tech Computer Engineering
Semester   : 3rd Year
College    : St. Andrews Institute of Technology & Management

=== Code Execution Successful ===
```

```
META-INF/
META-INF/MANIFEST.MF
MyDetails.class
```

2) Create an executable JAR (manifest + jar)

Create a manifest file or echo it:
echo "Main-Class: MyDetails" > manifest.txt

Create the jar:
jar cfm MyDetails.jar manifest.txt MyDetails.class

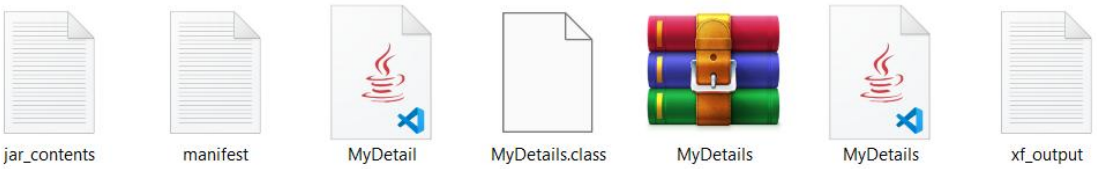
Run the jar:
java -jar MyDetails.jar

Run the commands to get tf output — “table of files” inside the JAR:
jar cfm MyDetails.jar manifest.txt
jar tf MyDetails.jar > jar_contents.txt

4)What you will see in the folder

After compiling and creating jar:

MyDetails.java
MyDetails.class
Manifest.txt
MyDetails.jar
Jar_contents (tf output)
Xf_output


jar_contents
manifest
MyDetail
MyDetails.class
MyDetails
MyDetails
xf_output

PROGRAM 2

Create a java program to implement stack and queue concepts.

Java Program

```
import java.util.LinkedList;

import java.util.Queue;

import java.util.S Output:tack;

import java.util.Scanner;

public class StackQueueDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // ----- Stack Implementation -----

        Stack<Integer> stack = new Stack<>();

        System.out.println("----- STACK OPERATIONS -----");

        stack.push(10);

        stack.push(20);

        stack.push(30);

        System.out.println("Stack after pushing elements: " + stack);

        stack.pop();

        System.out.println("Stack after one pop: " + stack);        System.out.println("Top
element in stack: " + stack.peek());
```

```
// ----- Queue Implementation -----

Queue<Integer> queue = new LinkedList<>();

System.out.println("\n----- QUEUE OPERATIONS -----");

queue.add(10);

queue.add(20);

queue.add(30);

System.out.println("Queue after adding elements: " + queue);

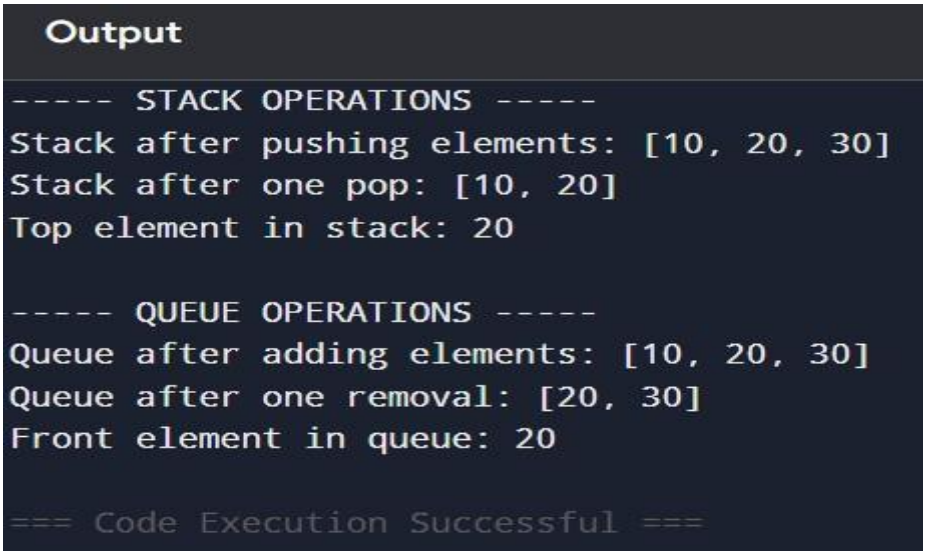
queue.remove();

System.out.println("Queue after one removal: " + queue);

System.out.println("Front element in queue: " + queue.peek());

sc.close(); }}
```

Output:



PROGRAM 3

Write a java package to show dynamic polymorphism and interfaces.

File 1 (Shape.java):

```
package geometry;

public interface Shape { void
    draw();
    double area();
}
```

File 2 (Circle.java):

```
package geometry;

public class Circle implements Shape { double
    radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public void draw() {
        System.out.println("Drawing a Circle");
    }

    public double area() {
        return 3.14 * radius * radius;
    }
}
```

File 3 (Rectangle.java):

```
package geometry;

public class Rectangle implements Shape { double
    length, breadth;

    public Rectangle(double length, double breadth) { this.length =
        length;
```

```
        this.breadth = breadth;
    }

    public void draw() { System.out.println("Drawing a
        Rectangle");
    }

    public double area() { return
        length * breadth;
    }
}
```

Main File (TestShapes.java):

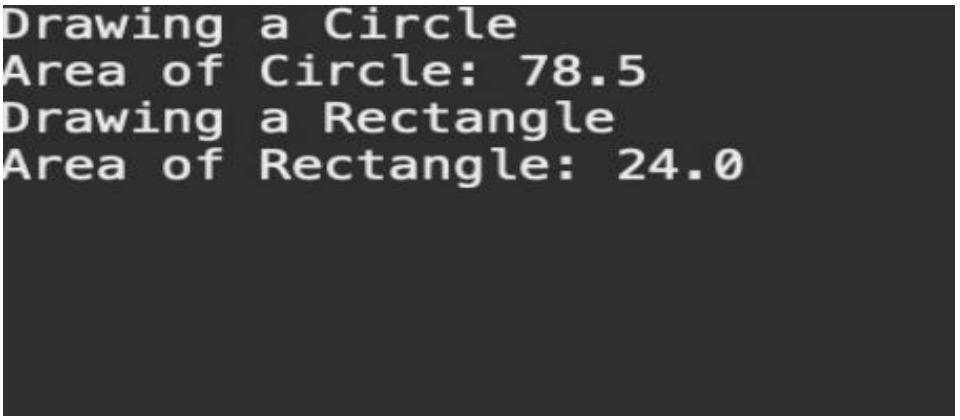
```
import geometry.Shape; import
geometry.Circle; import
geometry.Rectangle;

public class TestShapes {
    public static void main(String[] args) { Shape
        shape;

        shape = new Circle(5.0);
        shape.draw();
        System.out.println("Area of Circle: " + shape.area());

        shape = new Rectangle(4.0, 6.0);
        shape.draw();
        System.out.println("Area of Rectangle: " + shape.area());
    }
}
```

Output:



PROGRAM 4

Write a program to implement the concept of String and

String Buffer Java Program

```
public class StringExample { public static void
    main(String[] args) { System.out.println("-----
        STRING OPERATIONS
                                                                    ")
        ;
        String str1 = "Hello";
        String str2 = "World";

        System.out.println("String 1: " + str1);
        System.out.println("String 2: " + str2);

        String str3 = str1.concat(" " + str2);
        System.out.println("Concatenated String: " + str3);

        System.out.println("Uppercase: " + str3.toUpperCase());
        System.out.println("Substring

        (0–5): " + str3.substring(0, 5)); System.out.println("Length of String: " +
        str3.length());

        String replaced = str3.replace("World", "Java");
        System.out.println("After replace: " + replaced);

         System.out.println("\n----- STRINGBUFFER OPERATIONS                ");
        StringBuffer sb = new StringBuffer("Java");

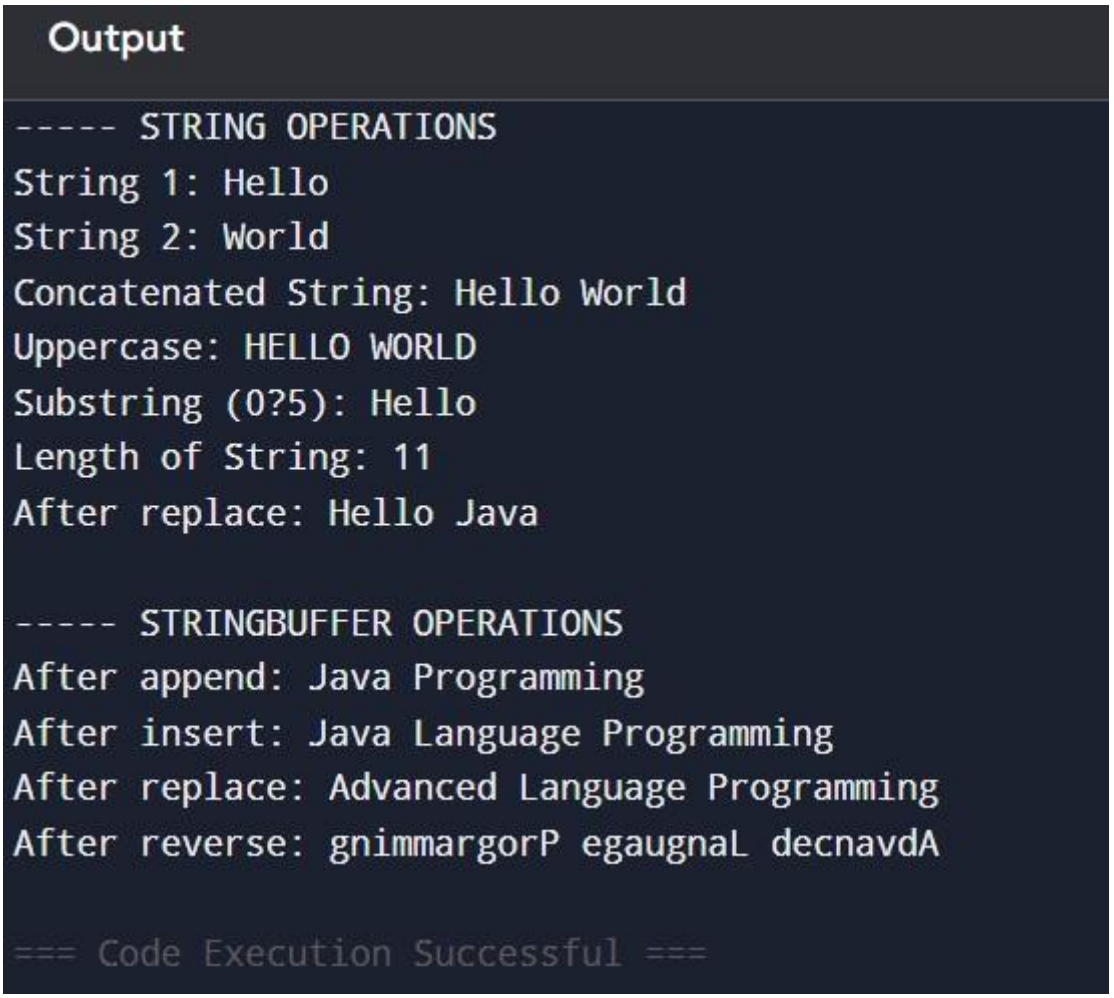
        sb.append(" Programming"); System.out.println("After
        append: " + sb);

        sb.insert(4, " Language");
        System.out.println("After insert: " + sb);

        sb.replace(0, 4, "Advanced");
        System.out.println("After replace: " + sb);

        sb.reverse();
        System.out.println("After reverse: " + sb);
        sb.reverse(); // Re-reverse to original
        sb.delete(4, 13);
        System.out.println("After delete: " + sb);
    }
}
```

Output:


Output
----- STRING OPERATIONS
String 1: Hello
String 2: World
Concatenated String: Hello World
Uppercase: HELLO WORLD
Substring (0?5): Hello
Length of String: 11
After replace: Hello Java
----- STRINGBUFFER OPERATIONS
After append: Java Programming
After insert: Java Language Programming
After replace: Advanced Language Programming
After reverse: gnimmargorP egaugnaL decnavdA
=== Code Execution Successful ===

PROGRAM 5

Write a program in java to implement the concept of Exception Handling with Five Keywords :- TRY,CATCH,FINALLY,THROW AND THROWS.

Java Program:

```
// Filename:
ExceptionHandlingDemo.java import
java.util.Scanner;

public class ExceptionHandlingDemo {

   static void divideNumbers(int a, int b) throws
ArithmeticException {        if (b == 0) {            throw new
ArithmeticException("Division by zero is not allowed!");
      } else {
int result = a /
b;
         System.out.println("Result: " + result);
      }
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

t
r
y
{
         System.out.println("----- EXCEPTION HANDLING DEMO --
---");          System.out.print("Enter first number: ");          int
num1 = sc.nextInt();

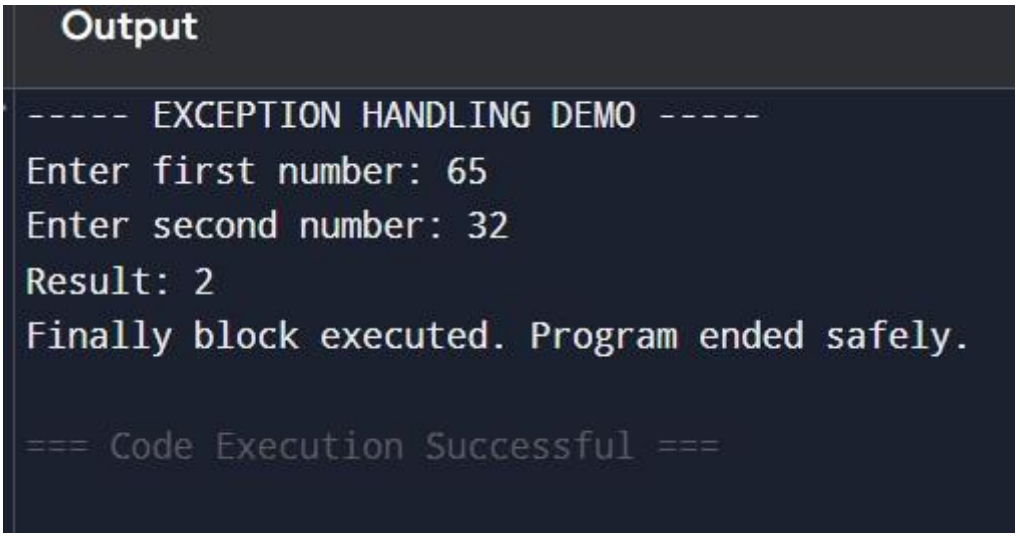
         System.out.print("Enter second
number: ");          int num2 = sc.nextInt();

         divideNumbers(num1,
num2);       } catch
(ArithmeticException e) {
         System.out.println("Exception Caught: " + e.getMessage());
      } finally {
         System.out.println("Finally block executed. Program ended safely.");
      }

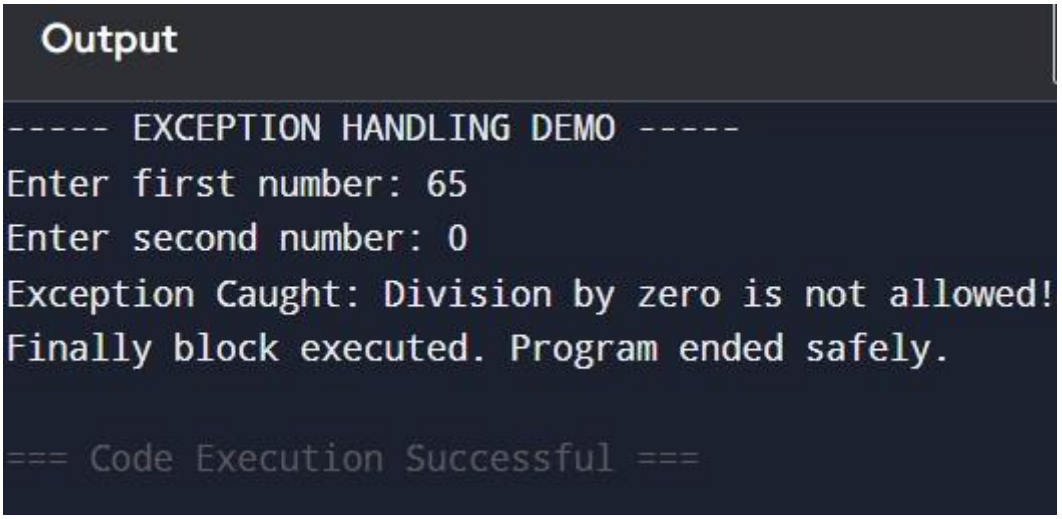
      sc.close();
   }
}
```

Output:

Normal Case:



Exception Case:



PROGRAM 6

Write a java program to show the concept of multithreading.

Java Program:

```
class ThreadA extends Thread { public
    void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread A: " + i); try {
                Thread.sleep(500); // pause for half second
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
        System.out.println("Thread A finished execution.\n");
    }
}

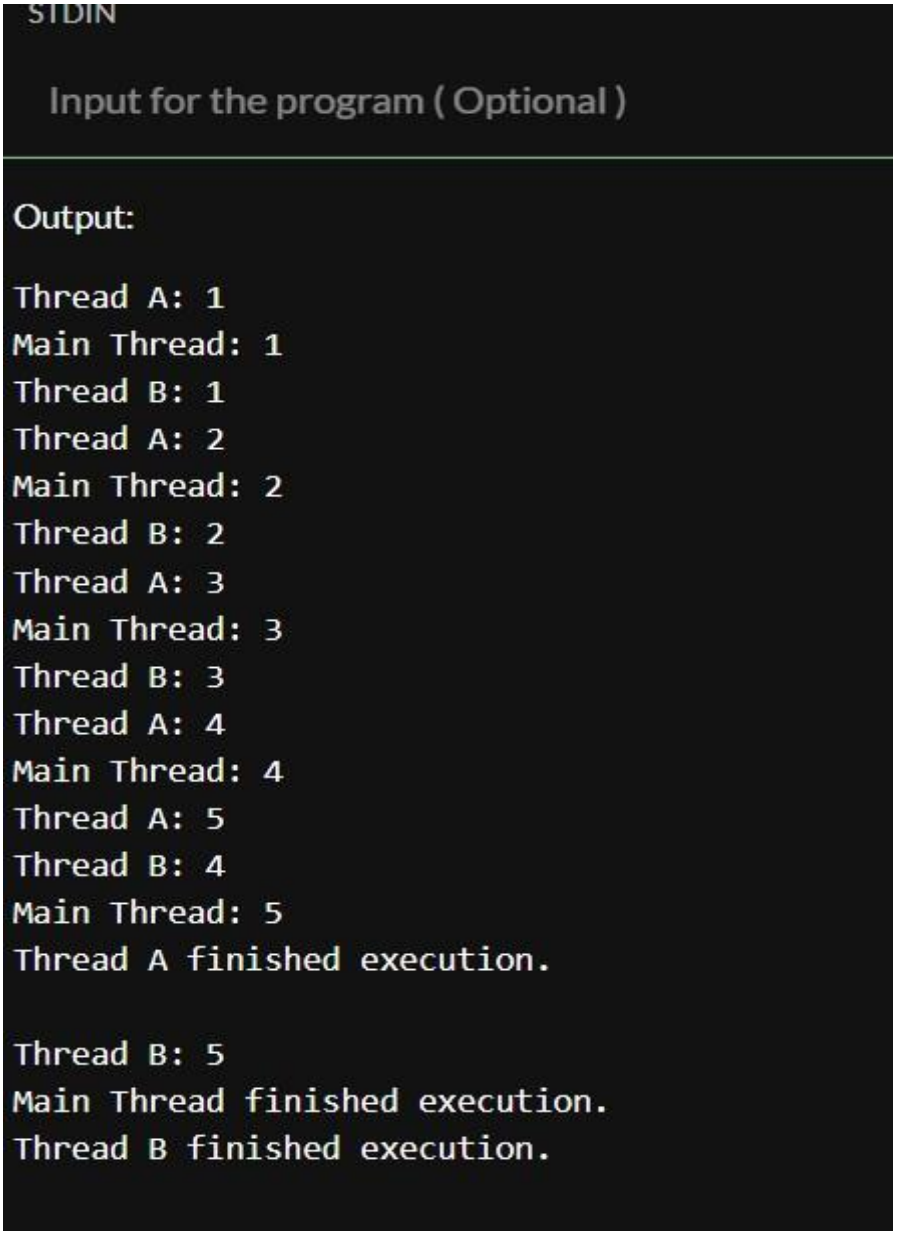
class ThreadB implements Runnable { public
    void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread B: " + i); try {
                Thread.sleep(700); // pause for 0.7 second
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
        System.out.println("Thread B finished execution.\n");
    }
}

public class MultithreadingDemo { public
    static void main(String[] args) {
        ThreadA t1 = new ThreadA();
        ThreadB tb = new ThreadB();
        Thread t2 = new Thread(tb);
        t1.start();
```

```
        t2.start();

        for (int i = 1; i <= 5; i++) {
            System.out.println("Main Thread: " + i); try {
                Thread.sleep(600);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
        System.out.println("Main Thread finished execution.");
    }
}
```

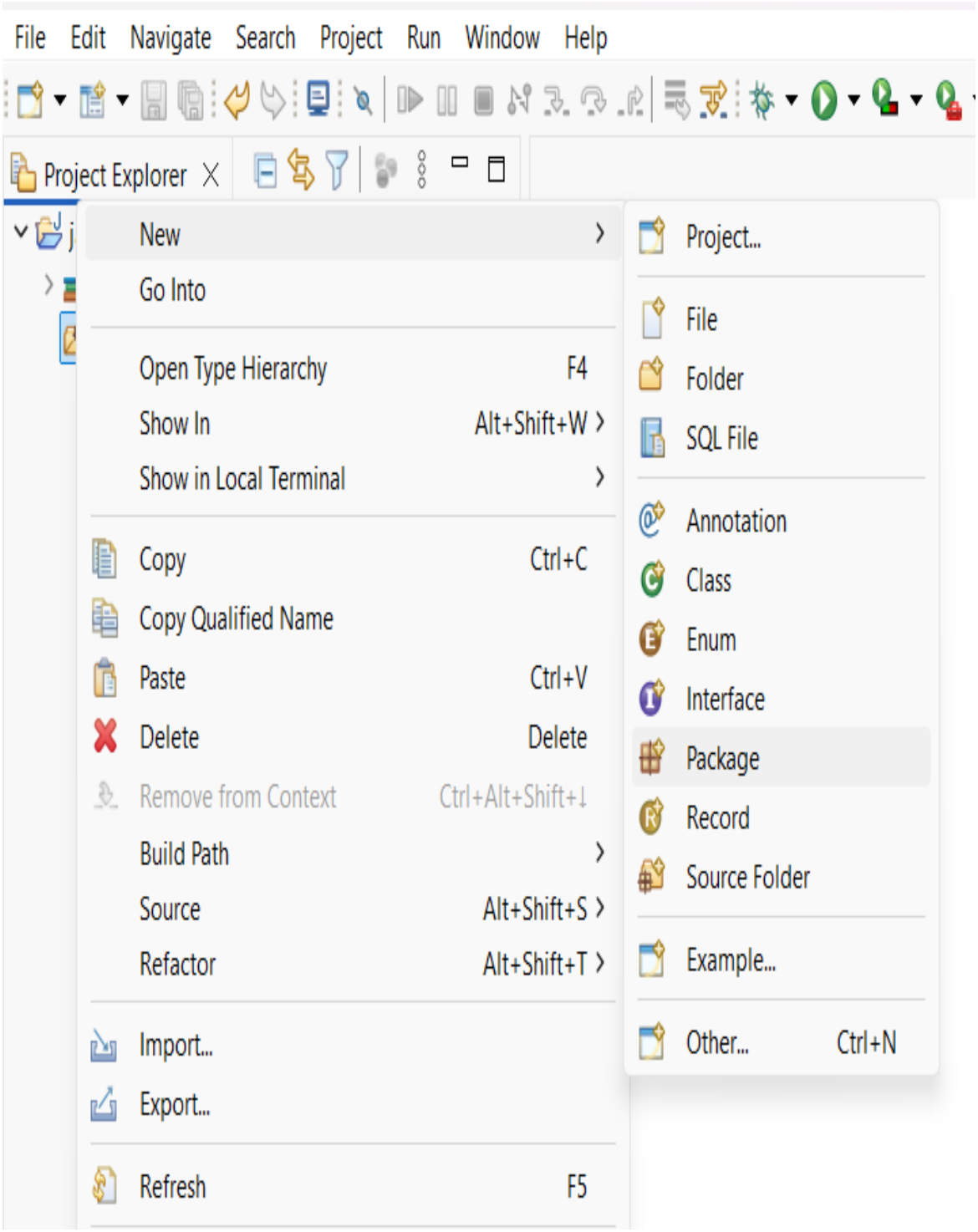
Output:



PROGRAM 7

WAP in java to create packages in java.

Java Program:



File 1: MyPackageClass.java

```
package mypackage;

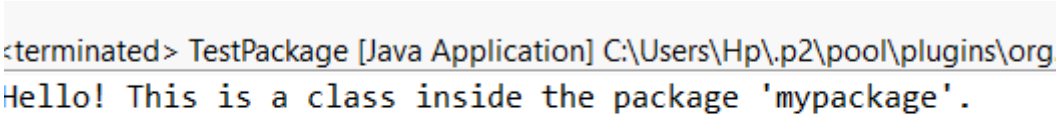
public class MyPackageClass { public void displayMessage() {
        System.out.println("Hello! This is a class inside the package 'mypackage'.");
    }
}
```

File 2: TestPackage.java

```
import mypackage.MyPackageClass;

public class TestPackage {
    public static void main(String[] args) { MyPackageClass obj = new MyPackageClass();
    obj. displayMessage();
```

Output:



PROGRAM 8

WAP in Java to create a Calculator using AWT. Java Program:

```
package mypackage;

import javax.swing.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {
   JTextField t1, t2, t3;
   JButton b1, b2, b3, b4;

   public Calculator() {
      setTitle("Priyavart CALCULATOR");
      setSize(350, 300);
      getContentPane().setLayout(null);

      JLabel l1 = new JLabel("Num 1:");
      l1.setBounds(50, 50, 80, 30);
      getContentPane().add(l1);
      t1 = new JTextField();
      t1.setBounds(130, 50, 150, 30);
      getContentPane().add(t1);

      JLabel l2 = new JLabel("Num 2:");
      l2.setBounds(50, 90, 80, 30);
      getContentPane().add(l2);
      t2 = new JTextField();
      t2.setBounds(130, 90, 150, 30);
      getContentPane().add(t2);

      JLabel l3 = new JLabel("Result:");
      l3.setBounds(50, 130, 80, 30);
      getContentPane().add(l3);
      t3 = new JTextField();
      t3.setBounds(130, 130, 150, 30);
      t3.setEditable(false);
      getContentPane().add(t3);

      b1 = new JButton("+");
      b1.setBounds(50, 180, 50, 30);
      b1.addActionListener(this);
      getContentPane().add(b1);

      b2 = new JButton("-");
      b2.setBounds(110, 180, 50, 30);
      b2.addActionListener(this);
      getContentPane().add(b2);

      b3 = new JButton("*");
      b3.setBounds(170, 180, 50, 30);

      b3.addActionListener(this);
```

```
        getContentPane().add(b3);

        b4 = new JButton("/");
        b4.setBounds(230, 180, 50, 30);
        b4.addActionListener(this);
        getContentPane().add(b4);

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            double a = Double.parseDouble(t1.getText());
            double b = Double.parseDouble(t2.getText());
            double res = 0;

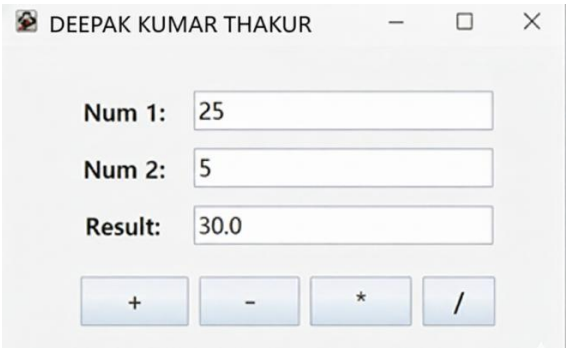
            if (e.getSource() == b1) res = a + b;
            else if (e.getSource() == b2) res = a - b;
            else if (e.getSource() == b3) res = a * b;
            else if (e.getSource() == b4) res = a / b;

            t3.setText(String.valueOf(res));
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Invalid Input!");
        }
    }

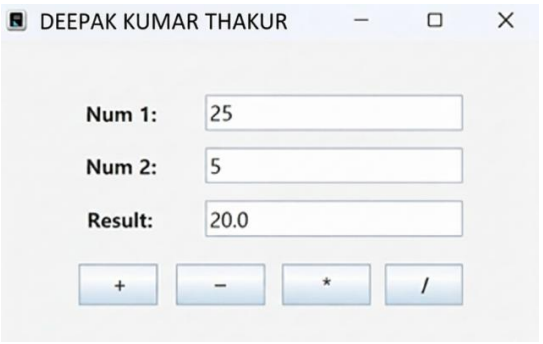
    public static void main(String[] args) {
        new Calculator();
    }
}
```

Output:

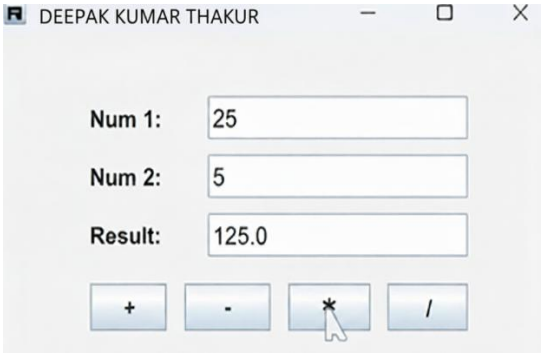
ADDITION



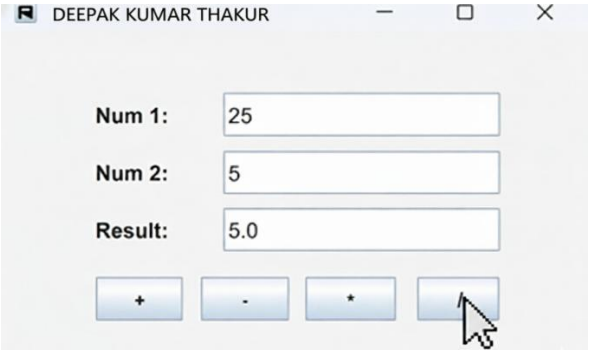
SUBTRACTION



MULTIPLICATION



DIVISION



PROGRAM 9

Write a program to Create an editor like MS-word using swings.

```
package editor;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;

public class MsEditor extends JFrame implements ActionListener, ComponentListener,
MouseListener {
   JTextArea area;
   JMenuItem n, o, s, e;
   JFileChooser fc;
   File f;

   public MsEditor() {
         setBackground(new Color(128, 255, 128));
      setTitle("Priyavart Word Editor");
      setSize(700, 500);
      getContentPane().setLayout(new BorderLayout());
      setDefaultCloseOperation(EXIT_ON_CLOSE);

      area = new JTextArea();
      area.setBackground(new Color(128, 255, 255));
      getContentPane().add(new JScrollPane(area), BorderLayout.CENTER);

      JMenuBar mb = new JMenuBar();
      JMenu m = new JMenu("File");
      n = new JMenuItem("New");
      o = new JMenuItem("Open");
      s = new JMenuItem("Save");
      e = new JMenuItem("Exit");
      m.add(n); m.add(o); m.add(s); m.add(e);
      mb.add(m);
      setJMenuBar(mb);

      fc = new JFileChooser();
      n.addActionListener(this);
      o.addActionListener(this);
      s.addActionListener(this);
      e.addActionListener(this);
      addComponentListener(this);
      area.addMouseListener(this);

      setVisible(true);
   }
```

```
public void actionPerformed(ActionEvent ae) {
    String cmd = ae.getActionCommand();
    System.out.println("Action performed: " + cmd);

    if (cmd.equals("New")) {
        area.setText("");
        f = null;
        System.out.println("New file created.");
    } else if (cmd.equals("Open")) {
        int i = fc.showOpenDialog(this);
        if (i == JFileChooser.APPROVE_OPTION) {
            f = fc.getSelectedFile();
            try (BufferedReader br = new BufferedReader(new FileReader(f))) {
                area.setText("");
                String line;
                while ((line = br.readLine()) != null)
                    area.append(line + "\n");
                System.out.println("Opened file: " + f.getAbsolutePath());
            } catch (Exception ex) {
                System.out.println("Error opening file: " + ex);
            }
        }
    } else if (cmd.equals("Save")) {
        if (f == null) {
            int i = fc.showSaveDialog(this);
            if (i == JFileChooser.APPROVE_OPTION)
                f = fc.getSelectedFile();
            else return;
        }
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(f))) {
            bw.write(area.getText());
            System.out.println("Saved file: " + f.getAbsolutePath());
        } catch (Exception ex) {
            System.out.println("Error saving file: " + ex);
        }
    } else if (cmd.equals("Exit")) {
        System.out.println("Exiting editor...");
        System.exit(0);
    }
}

public void componentResized(ComponentEvent e) {
    System.out.println("Window resized");
}

public void componentMoved(ComponentEvent e) {
    System.out.println("Window moved to");
}

public void componentShown(ComponentEvent e) {
    System.out.println("Window shown");
}
```

```
public void componentHidden(ComponentEvent e) {
        System.out.println("Window hidden");
    }




public void mouseEntered(MouseEvent e) {
        System.out.println("Mouse entered text area");
    }

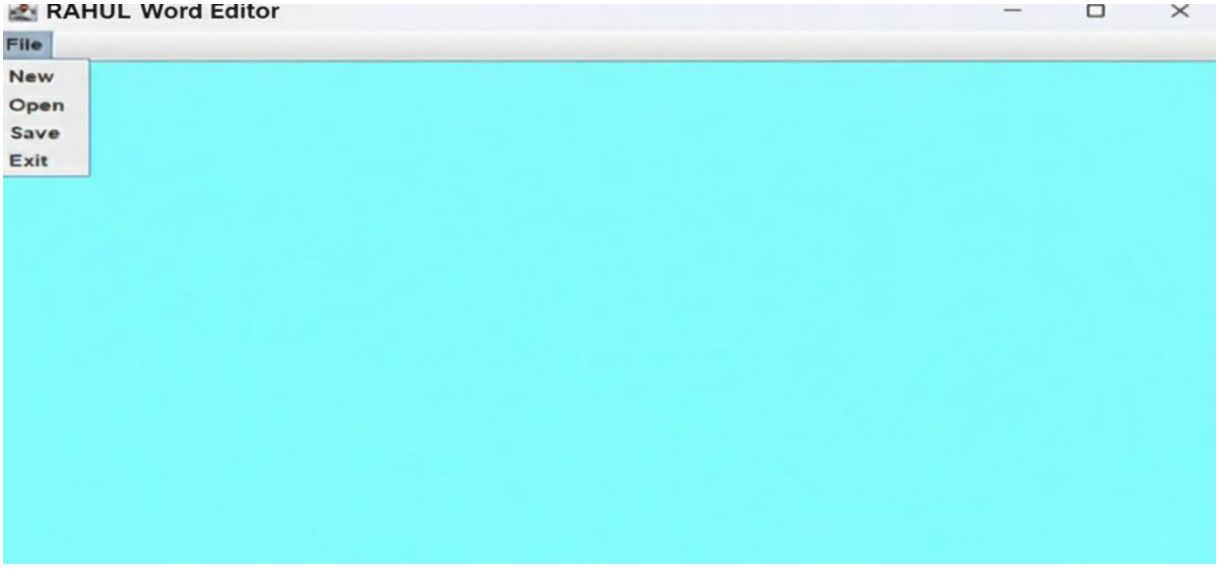
    public void mouseExited(MouseEvent e) {
        System.out.println("Mouse exited text area");
    }

    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse clicked inside text area");
    }

    public void mousePressed(MouseEvent e) {
        System.out.println("Mouse pressed");
    }

    public void mouseReleased(MouseEvent e) {
        System.out.println("Mouse released");
    }

    public static void main(String[] args) {
        new MsEditor();
    }
}
```



OUTPUT :

```
Window moved to
Window moved to
Window moved to
Mouse entered text area
Mouse exited text area
Mouse entered text area
Mouse exited text area
Window moved to
Window resized
Mouse entered text area
Mouse exited text area
```

PROGRAM 10

Write a java program to show the concept of Collections.

```
package mypackage;

import java.util.*;

public class coll1 {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("Apple");
      list.add("Banana");
      list.add("Mango");
      list.add("Orange");
      System.out.println("ArrayList: " + list);
      list.remove("Banana");
      System.out.println("After remove Banana: " + list);

      LinkedHashSet<String> lhs = new LinkedHashSet<>();
      lhs.add("Dog");
      lhs.add("Cat");
      lhs.add("Elephant");
      lhs.add("bat");
      System.out.println("\nLinkedHashSet: " + lhs);
      lhs.remove("Dog");
      System.out.println("After remove Dog: " + lhs);

      SortedSet<Integer> sset = new TreeSet<>();
      sset.add(40);
      sset.add(10);
      sset.add(30);
      sset.add(20);
      System.out.println("\nSortedSet: " + sset);
      System.out.println("First: " + sset.first());
      System.out.println("Last: " + sset.last());
      sset.remove(30);
      System.out.println("After remove 30: " + sset);

      HashMap<Integer, String> map = new HashMap<>();
      map.put(1, "One");
      map.put(2, "Two");
      map.put(3, "Three");
      map.put(4, "Four");
      System.out.println("\nHashMap: " + map);
      map.remove(2);
      System.out.println("After remove key 2: " + map);

   }
}
```

Output:

```
<terminated> coll1 [Java Application] C:\Users\Hp\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre
ArrayList: [Apple, Banana, Mango, Orange]
After remove Banana: [Apple, Mango, Orange]

LinkedHashSet: [Dog, Cat, Elephant, bat]
After remove Dog: [Cat, Elephant, bat]

SortedSet: [10, 20, 30, 40]
First: 10
Last: 40
After remove 30: [10, 20, 40]

HashMap: {1=One, 2=Two, 3=Three, 4=Four}
After remove key 2: {1=One, 3=Three, 4=Four}
```

PROGRAM 11

Write a program on synchronization using mutual exclusive types.

```
package editor;

class TicketBooking {
   int totalTickets = 5;

   synchronized void bookTicket(String name, int numberOfTickets) {
      if (totalTickets >= numberOfTickets) {
         System.out.println(name + " booked " + numberOfTickets + " ticket(s).");
         totalTickets -= numberOfTickets;
         System.out.println("Tickets left: " + totalTickets);
      } else {
         System.out.println("Sorry not enough tickets available.");
         System.out.println("Tickets left: " + totalTickets);
      }
   }
}

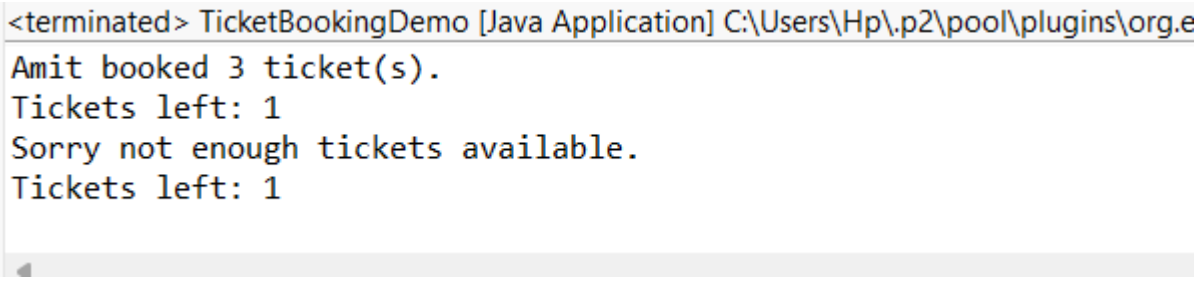
class BookingThread extends Thread {
   TicketBooking booking;
   String name;
   int tickets;

   BookingThread(TicketBooking booking, String name, int tickets) {
      this.booking = booking;
      this.name = name;
      this.tickets = tickets;
   }

   public void run() {
      booking.bookTicket(name, tickets);
   }
}

public class TicketBookingDemo {
   public static void main(String[] args) {
      TicketBooking obj = new TicketBooking();
      BookingThread t1 = new BookingThread(obj, "Nikhil", 2);
      BookingThread t2 = new BookingThread(obj, "Ravi", 1);
      BookingThread t3 = new BookingThread(obj, "Amit", 3);
      t1.start();
      t2.start();
      t3.start();
   }
}
```

Output:


<terminated> TicketBookingDemo [Java Application] C:\Users\Hp\.p2\pool\plugins\org.e
Amit booked 3 ticket(s).
Tickets left: 1
Sorry not enough tickets available.
Tickets left: 1